

Heap

Heap

A heap is a data structure that stores a collection of objects (with keys), and has the following properties:

- Complete Binary tree
- Heap Order

It is implemented as an array where each node in the tree corresponds to an element of the array.

Heap

- A heap is a complete binary tree except the bottom level adjusted to the left.
- The value of each node is greater than that of its two children. (Max Heap)
- The value of each node is less than that of its two children. (Min Heap)
- Height of the heap is $\log_2 n$.
- Example

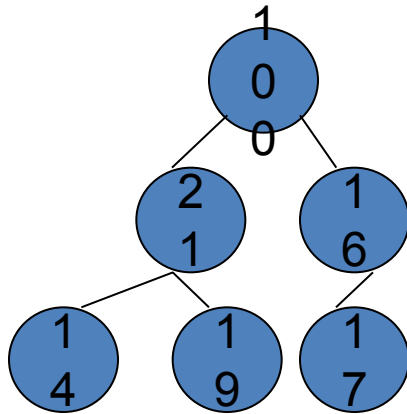


Figure: Not a Heap

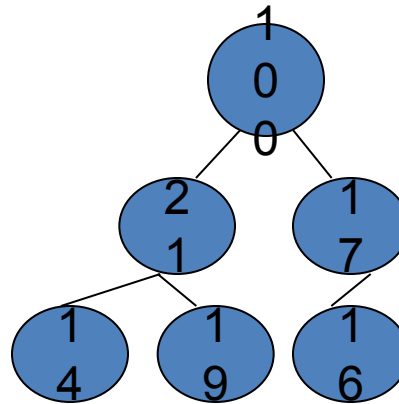
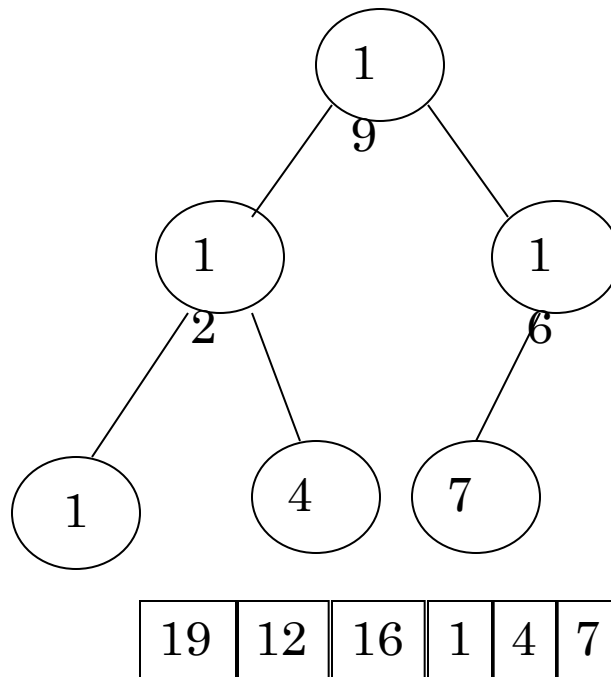


Figure: A Heap

Heap

- The binary heap data structure is an array that can be viewed as a complete binary tree. Each node of the binary tree corresponds to an element of the array.
- The array is completely filled on all levels except possibly lowest.



Array A

What is a Heap?

A heap is also known as a priority queue and can be represented by a binary tree with the following properties:

Structure property: A heap is a completely filled binary tree with the exception of the bottom row, which is filled from left to right

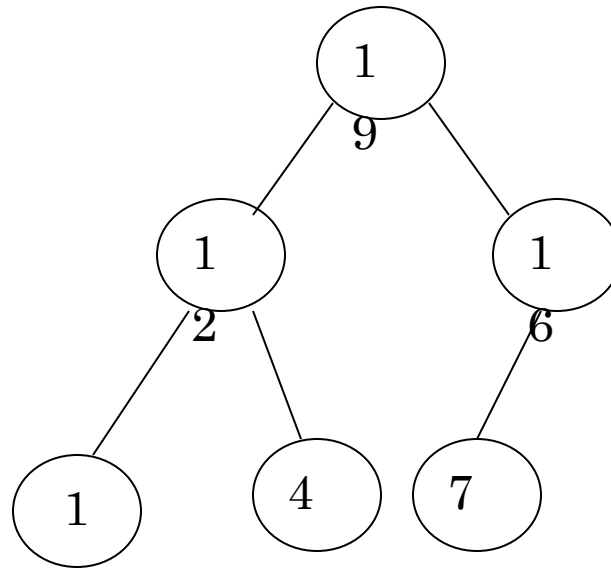
Heap Order property: For every node x in the heap, the parent of x greater than or equal to the value of x .

(known as a maxHeap).

Definition

- Max Heap
 - Store data in ascending order
 - Has property of
$$A[\text{Parent}(i)] \geq A[i]$$
- Min Heap
 - Store data in descending order
 - Has property of
$$A[\text{Parent}(i)] \leq A[i]$$

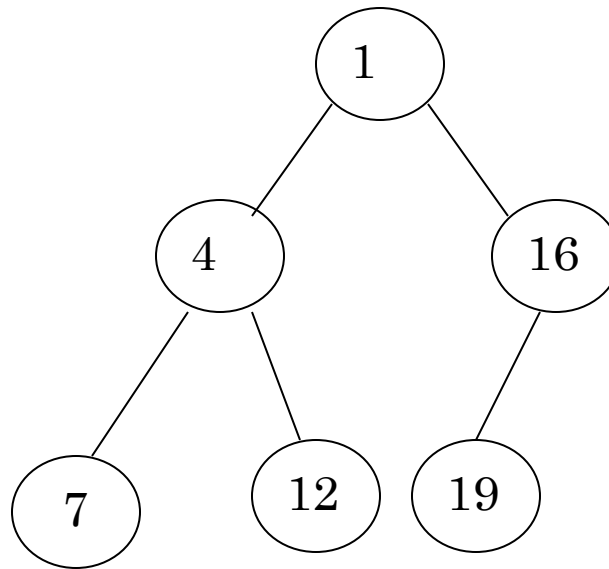
Max Heap Example



19	12	16	1	4	7
----	----	----	---	---	---

Array A

Min heap example



1	4	16	7	12	19
---	---	----	---	----	----

Array A

Heap

- The root of the tree $A[1]$ and given index i of a node, the indices of its parent, left child and right child can be computed

PARENT (i)

return $\text{floor}(i/2)$

LEFT (i)

return $2i$

RIGHT (i)

return $2i + 1$

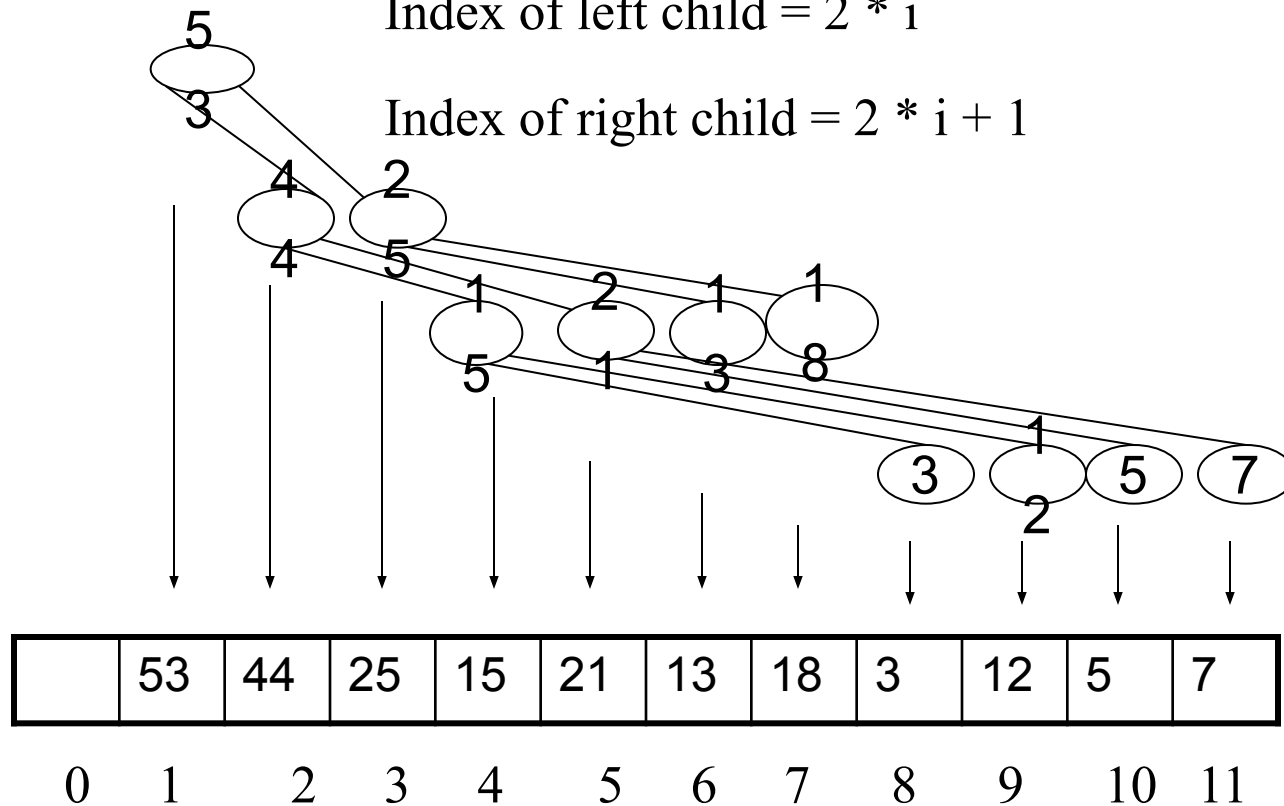
Array implementation

For any node i , the following formulas apply:

The index of its parent = $i / 2$

Index of left child = $2 * i$

Index of right child = $2 * i + 1$



Heap Implementation

- We can use an array (due to the regular structure or completeness of binary tree).
- For a node N with location i, the following factors can be calculated.
 1. Left child of N is in location $(2 * i)$.
 2. Right child of N is in location $(2 * i + 1)$.
 3. Parent of N is in location $[i/2]$.
- Example

1	2	3	4	5	6	7	8	9
10	21	17	14	19	16			
0								

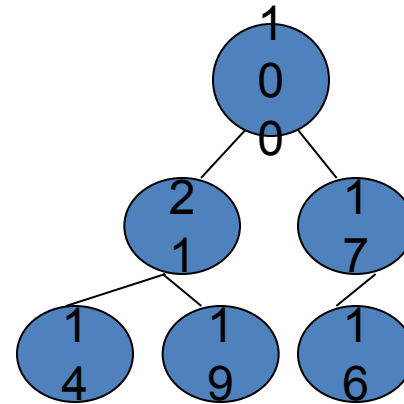


Figure: Heap and Its Array Representation

Insertion

□ Algorithm

1. Add the new element to the next available position at the lowest level
2. Restore the max-heap property if violated
 - General strategy is percolate up (or bubble up): if the parent of the element is smaller than the element, then interchange the parent and child.

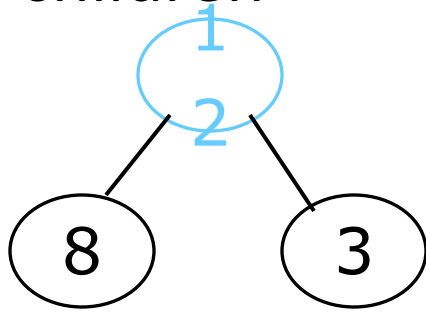
OR

Restore the min-heap property if violated

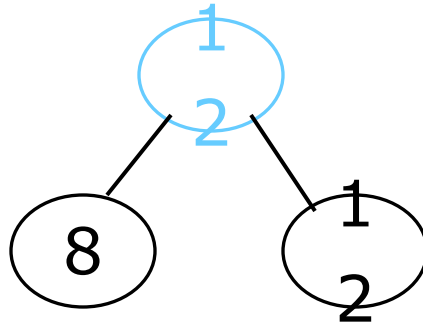
- General strategy is percolate up (or bubble up): if the parent of the element is larger than the element, then interchange the parent and child.

The heap property

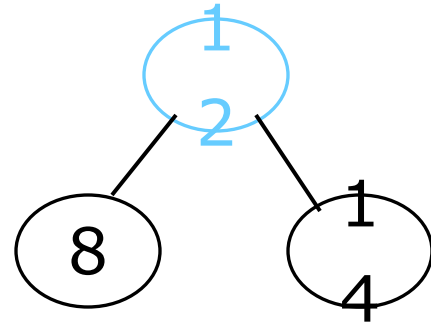
- A node has the heap property if the value in the node is as large as or larger than the values in its children



Blue node has
heap property



Blue node has
heap property

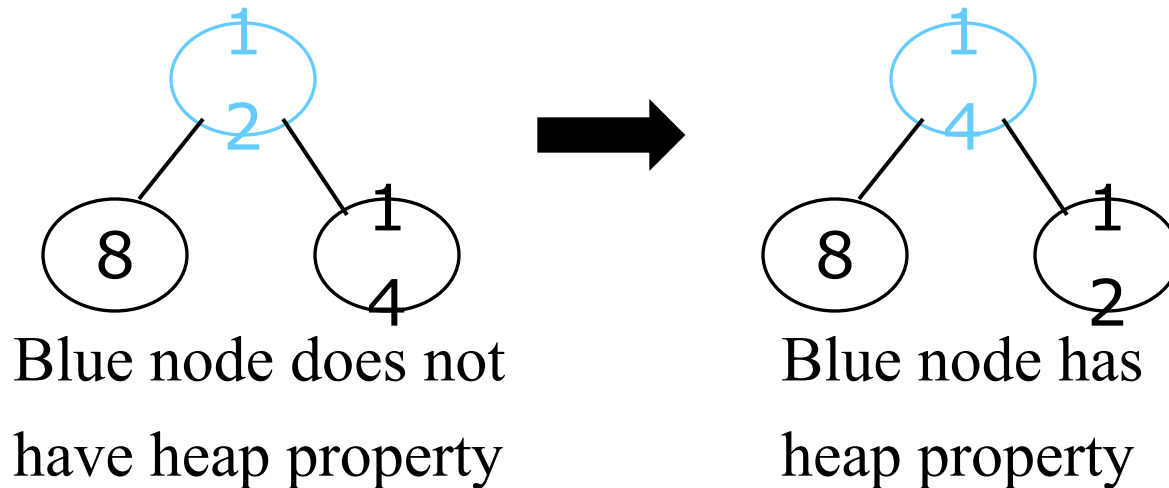


Blue node does not
have heap property

- All leaf nodes automatically have the heap property
- A binary tree is a heap if *all* nodes in it have the heap property

siftUp

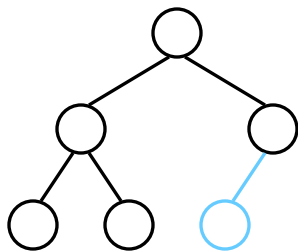
- Given a node that does not have the heap property, you can give it the heap property by exchanging its value with the value of the larger child



- This is sometimes called sifting up
- Notice that the child may have *lost* the heap property

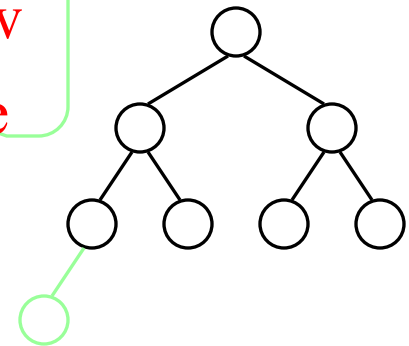
Constructing a heap I

- A tree consisting of a single node is automatically a heap
- We construct a heap by adding nodes one at a time:
 - Add the node just to the right of the rightmost node in the deepest level
 - If the deepest level is full, start a new level
- Examples:



Add a new
node here

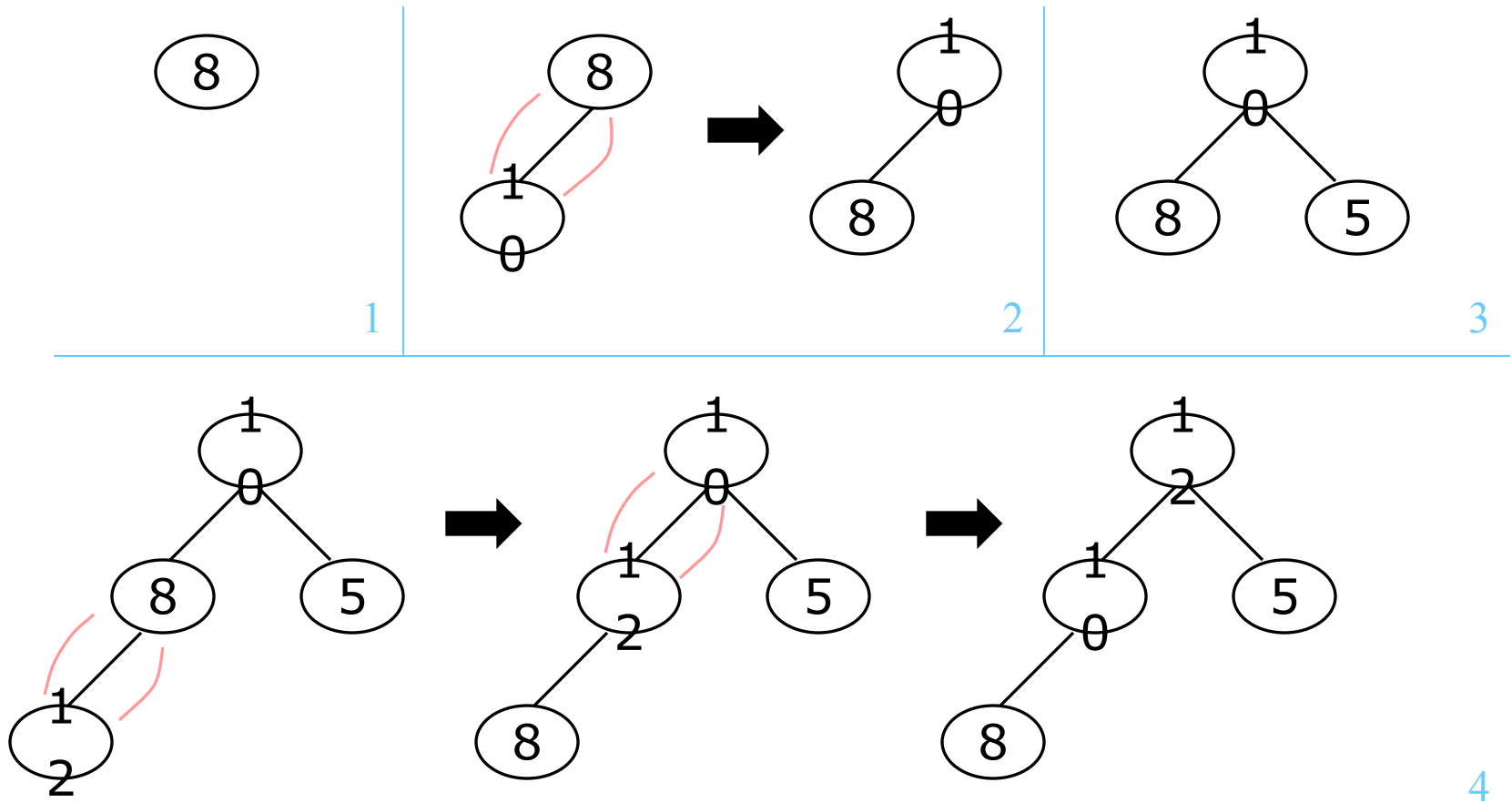
Add a new
node here



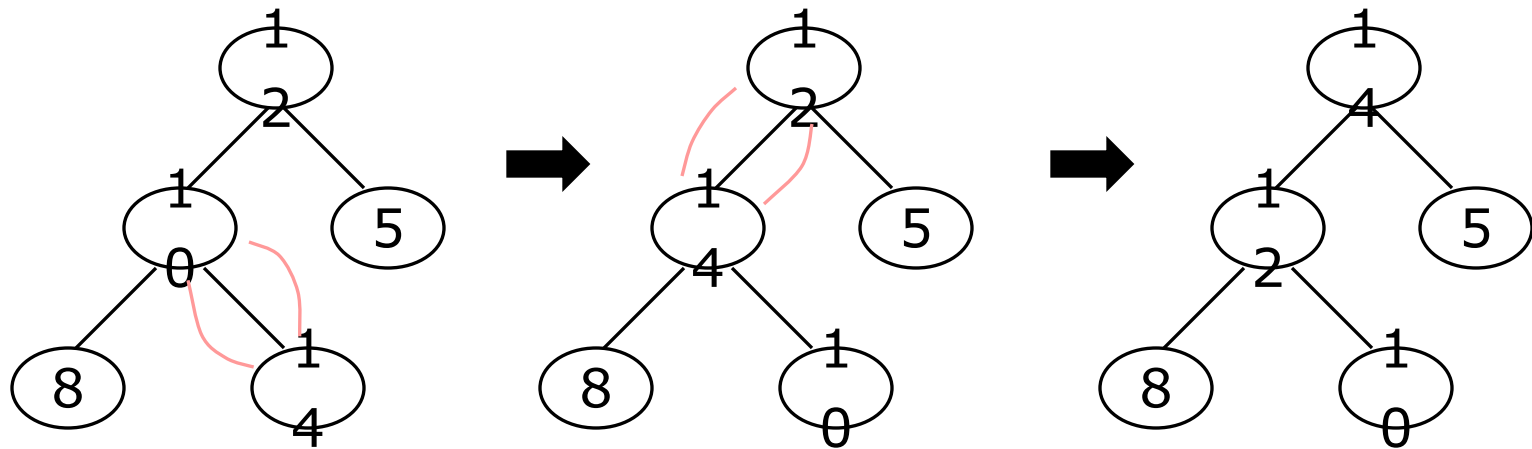
Constructing a heap II

- Each time we add a node, we may destroy the heap property of its parent node
- To fix this, we sift up
- But each time we sift up, the value of the topmost node in the sift may increase, and this may destroy the heap property of *its* parent node
- We repeat the sifting up process, moving up in the tree, until either
 - We reach nodes whose values don't need to be swapped (because the parent is *still* larger than both children), or
 - We reach the root

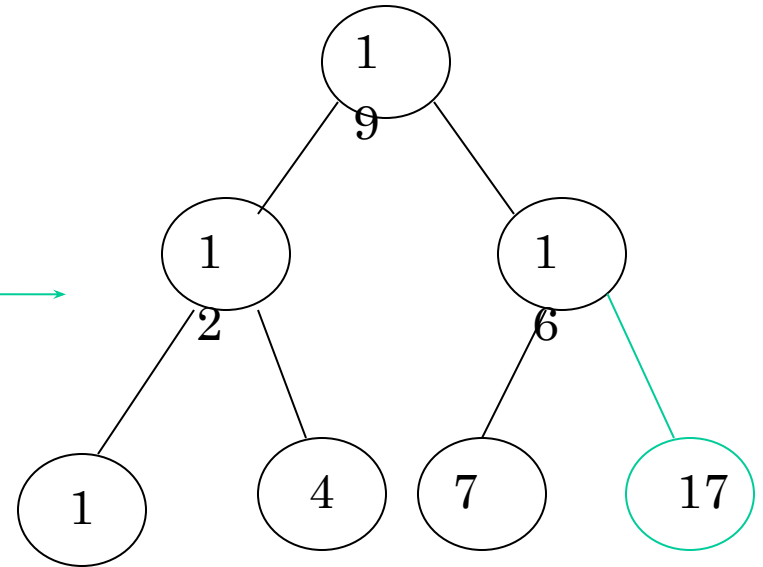
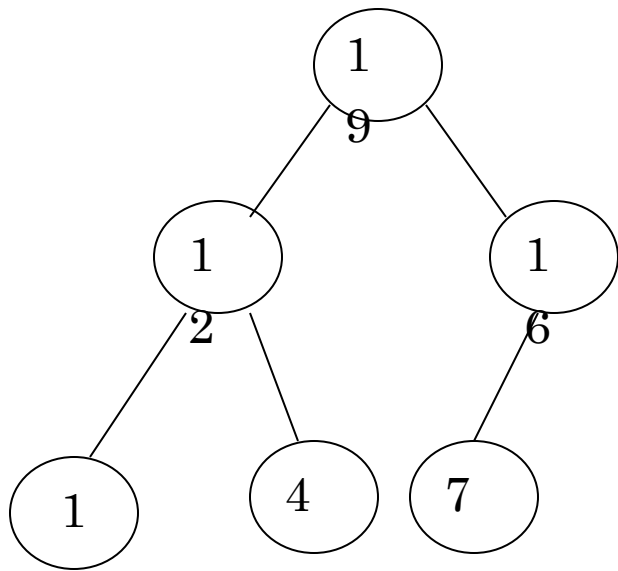
Constructing a heap III



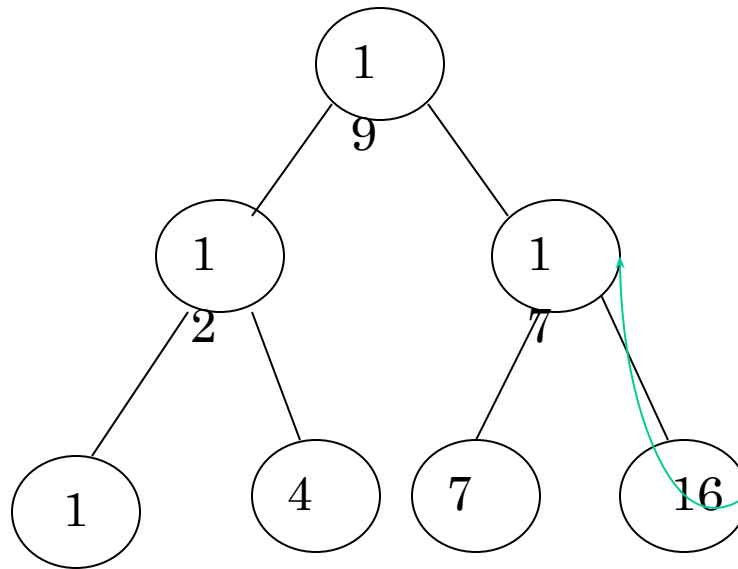
Other children are not affected



- The node containing 8 is not affected because its parent gets larger, not smaller
- The node containing 5 is not affected because its parent gets larger, not smaller
- The node containing 8 is still not affected because, although its parent got smaller, its parent is still greater than it was originally



Insert 17



swap

Percolate up to maintain the
heap property

Heap Operations

- (1) Insertion
- (2) Delete-max or delete-min
- (3) Make-heap (organize an arbitrary array as a heap)

Insertion

- Find the left-most open position and insert the new element NewElement.
- while (NewElement > Its Parent) exchange them.
- Example:

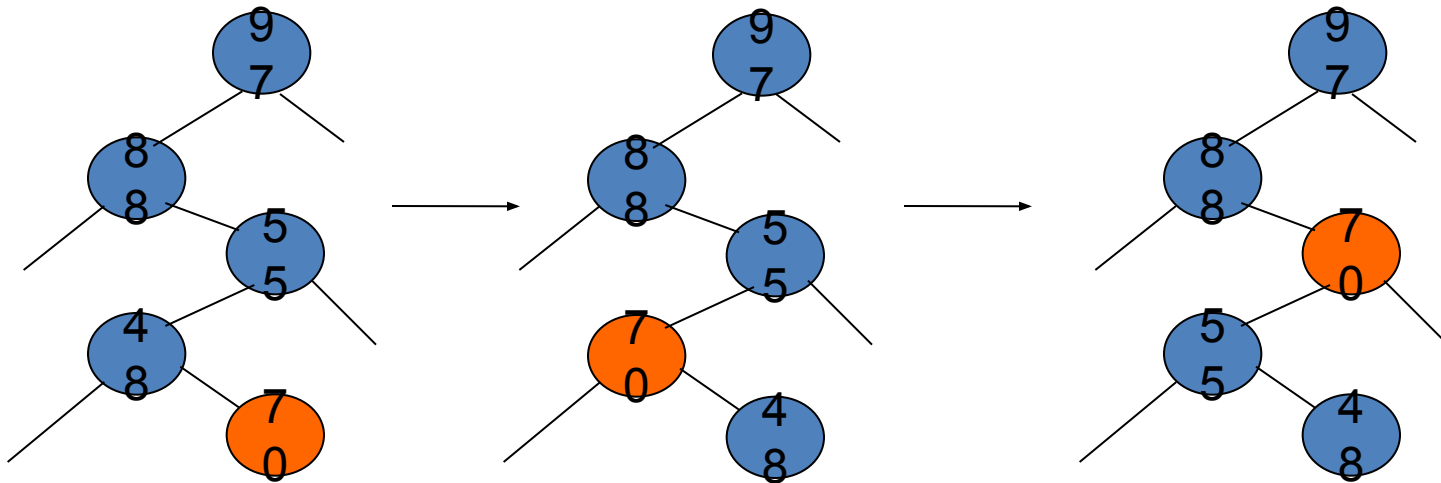


Figure: Item 70 is inserted.

Algorithm: INSHEAP(Heap, N, Item)

1. Set $N := N + 1$ and $Ptr := N$
2. Repeat steps 3 to 6 while $Ptr > 1$
3. Set $Par := Ptr/2$
4. If $Item < Heap[Par]$, then
Set $Heap[Ptr] := Item$ and Return.
1. Set $Heap[Ptr] := Heap[Par]$
2. Set $Ptr := Par$
3. $Heap[1] := Item$
4. Return.

Here,

Heap = Max heap

N = Total no. of items in the array

Item = New item to be inserted

The Complexity Analysis of Insert Operation

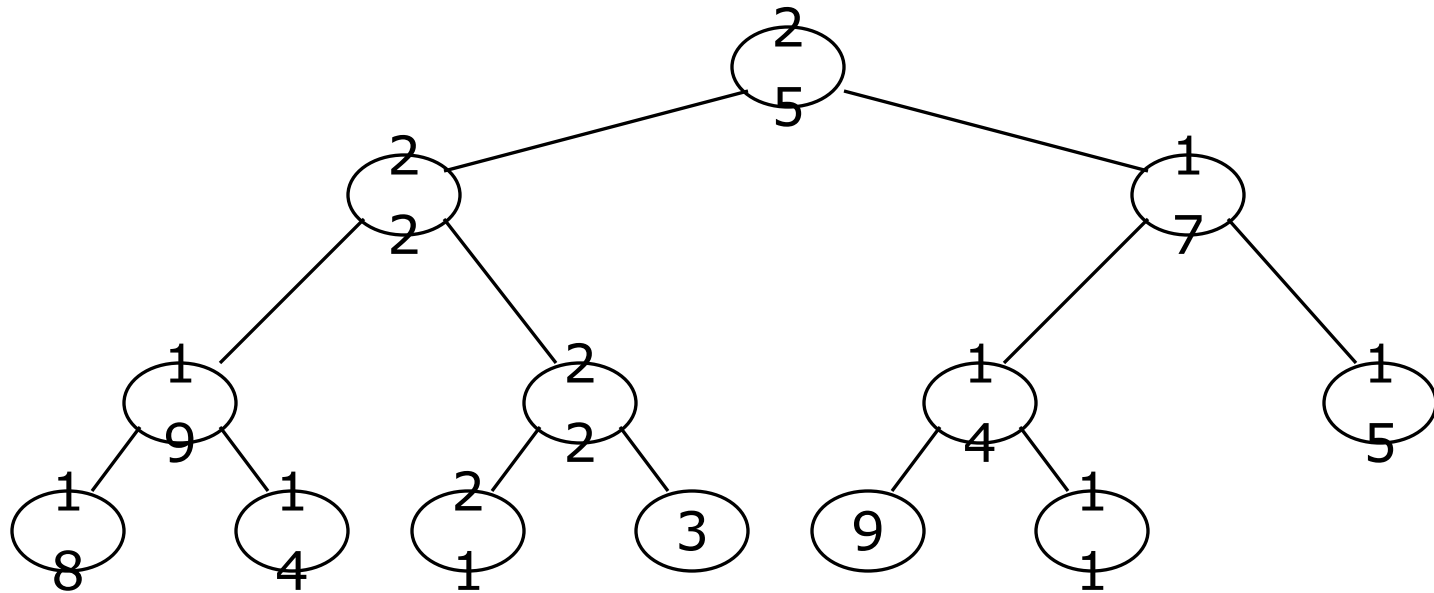
- Insertion takes $O(\log_2 n)$ times.
- Reasons:
 - 1) No. of operations = $O(\text{height of the heap})$
 - 2) Height of the heap: $O(\log_2 n)$

Deletion

- Delete max
 - Copy the last number to the root (overwrite the maximum element stored there).
 - Restore the max heap property by percolate down.
- Delete min
 - Copy the last number to the root (overwrite the minimum element stored there).
 - Restore the min heap property by percolate down.

A sample heap

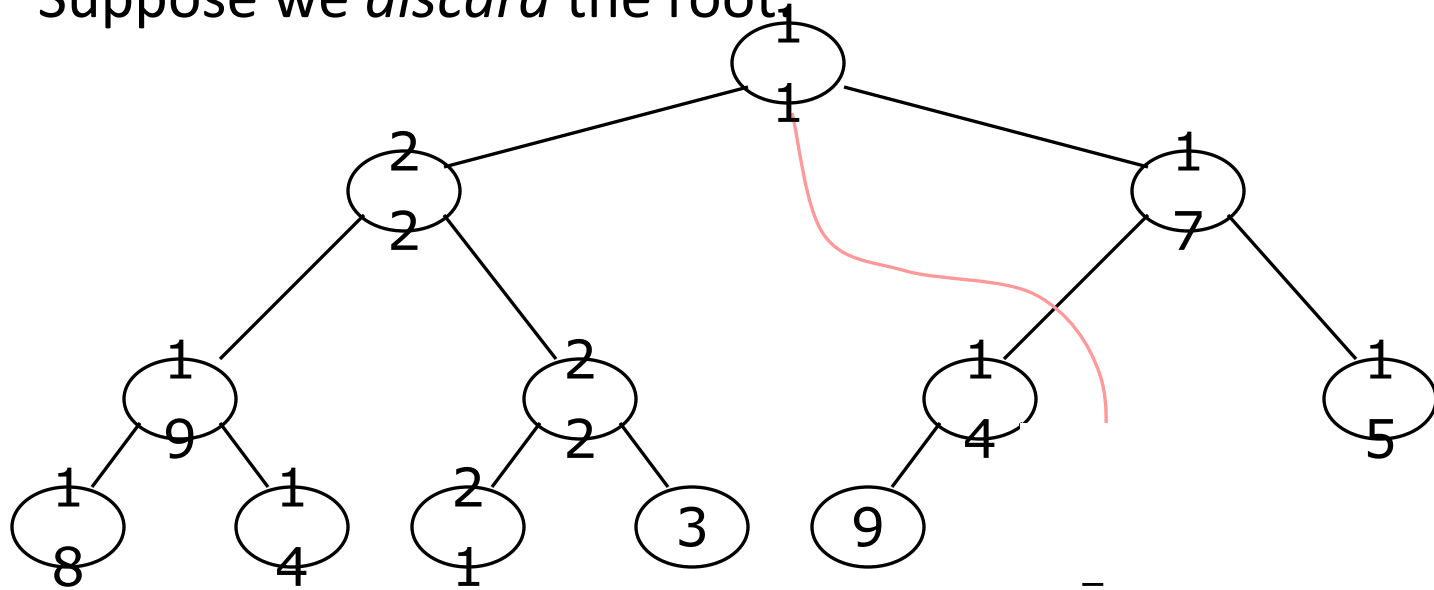
- Here's a sample binary tree after it has been heapified



- Notice that heapified does *not* mean sorted
- Heapifying does *not* change the shape of the binary tree; this binary tree is balanced and left-justified because it started out that way

Removing the root

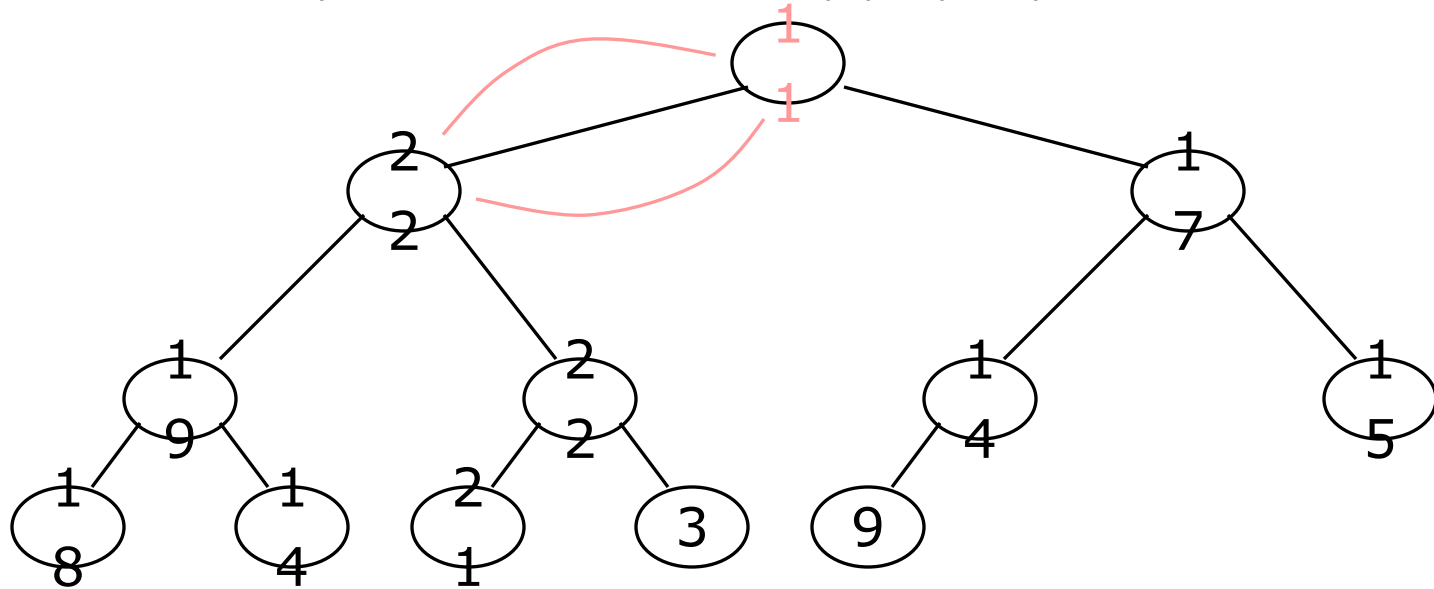
- Notice that the largest number is now in the root
- Suppose we *discard* the root;



- How can we fix the binary tree so it is once again *balanced and left-justified*?
- Solution: remove the rightmost leaf at the deepest level and use it for the new root

The **reHeap** method I

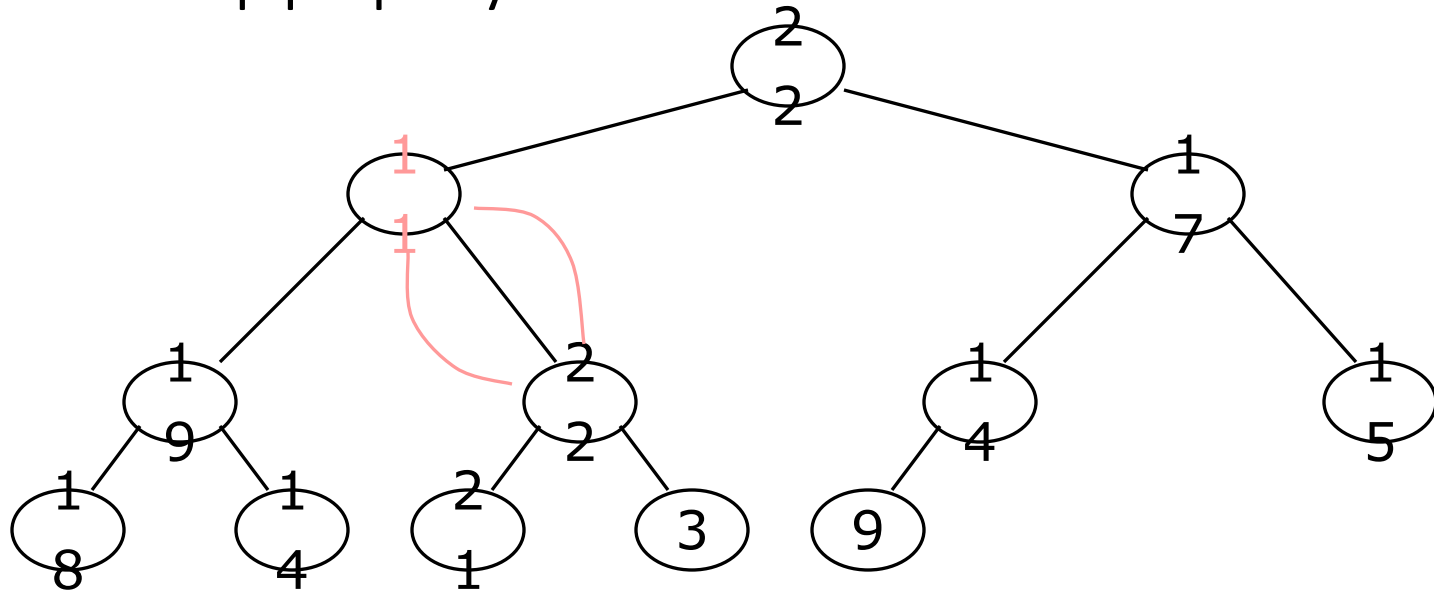
- Our tree is balanced and left-justified, but no longer a heap
- However, *only the root* lacks the heap property



- We can **siftUp()** the root
- After doing this, one and only one of its children may have lost the heap property

The reHeap method II

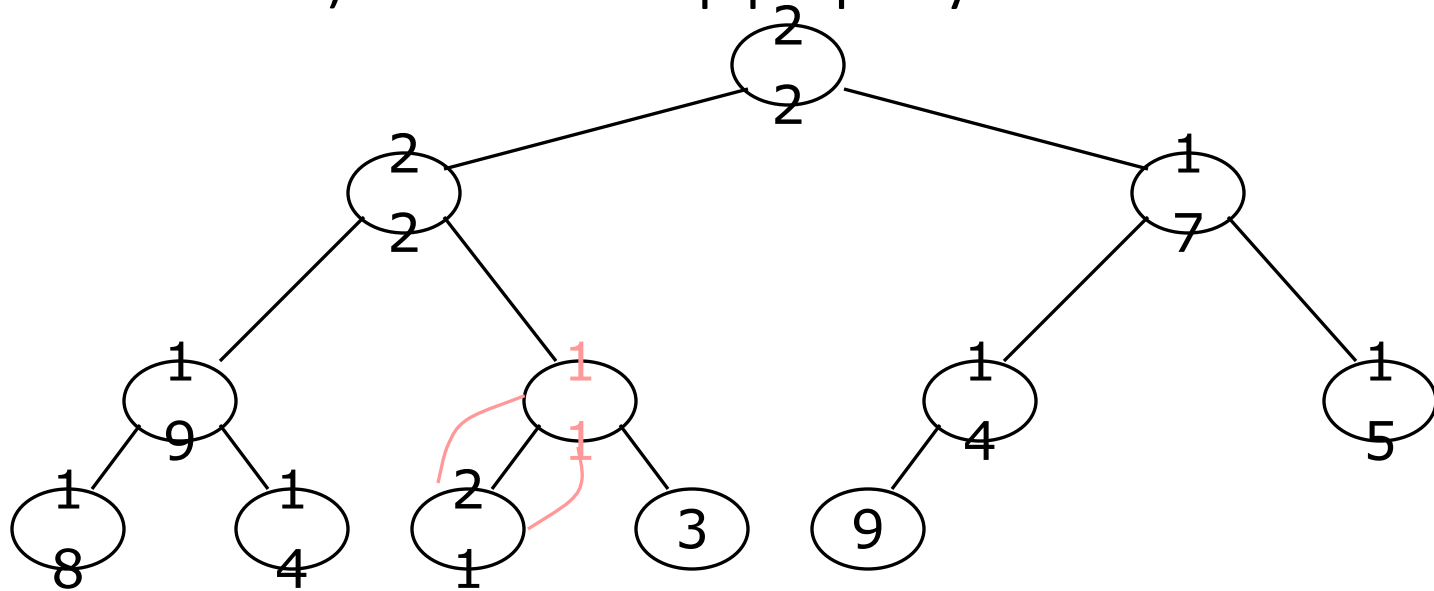
- Now the left child of the root (still the number 11) lacks the heap property



- We can siftUp() this node
- After doing this, one and only one of its children may have lost the heap property

The reHeap method III

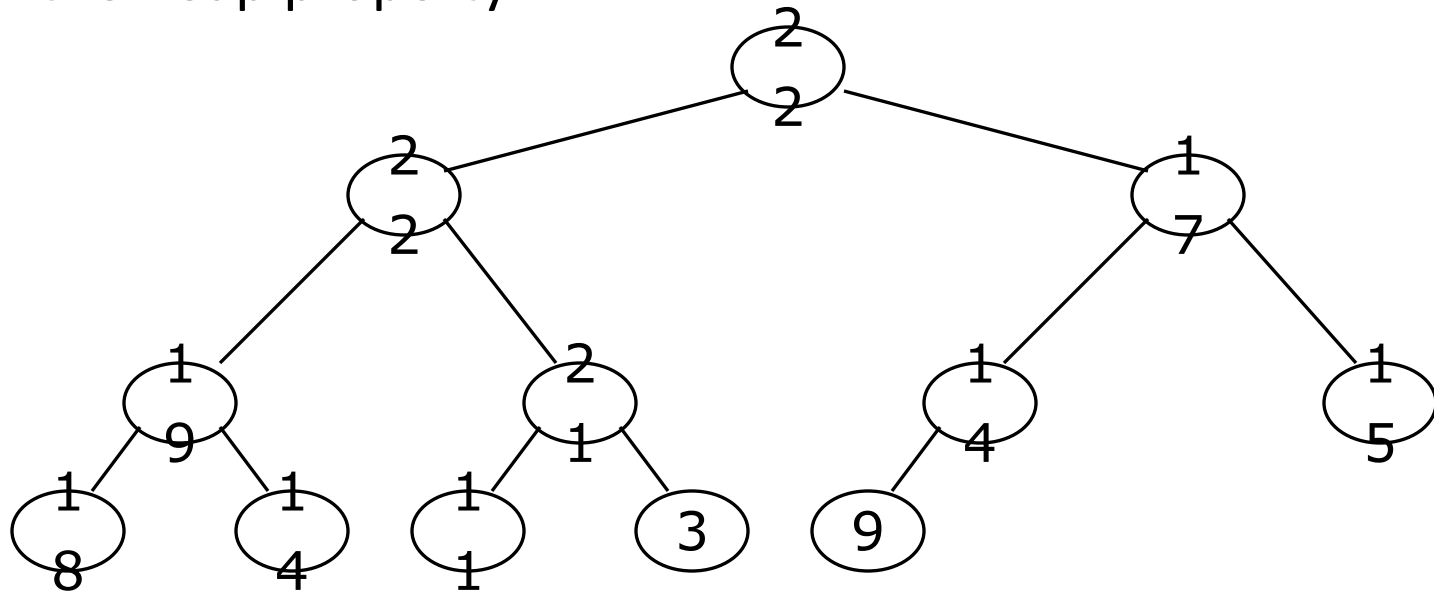
- Now the right child of the left child of the root (still the number 11) lacks the heap property:



- We can `siftUp()` this node
- After doing this, one and only one of its children may have lost the heap property —but it doesn't, because it's a leaf

The reHeap method IV

- Our tree is once again a heap, because every node in it has the heap property



- Once again, the largest (or *a* largest) value is in the root
- We can repeat this process until the tree becomes empty
- This produces a sequence of values in order largest to smallest

Delete-Max

- Replace the root node by the last node at bottom level maintaining completeness.
- While (new root < its children) exchange it with the greater of its children.
- Example:

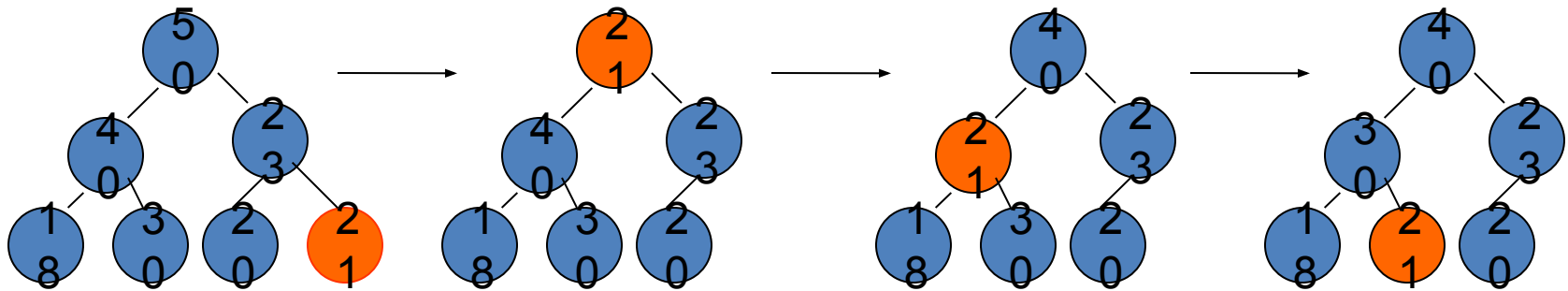


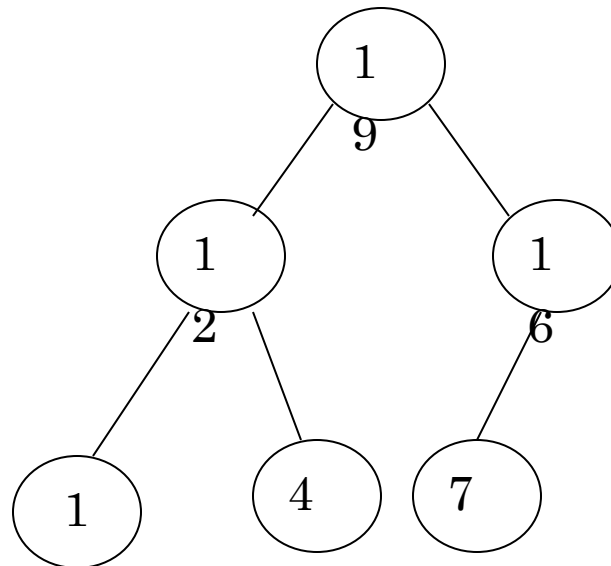
Figure: Item 50 (root) is deleted.

Algorithm: DELHEAP(Heap, N, Item)

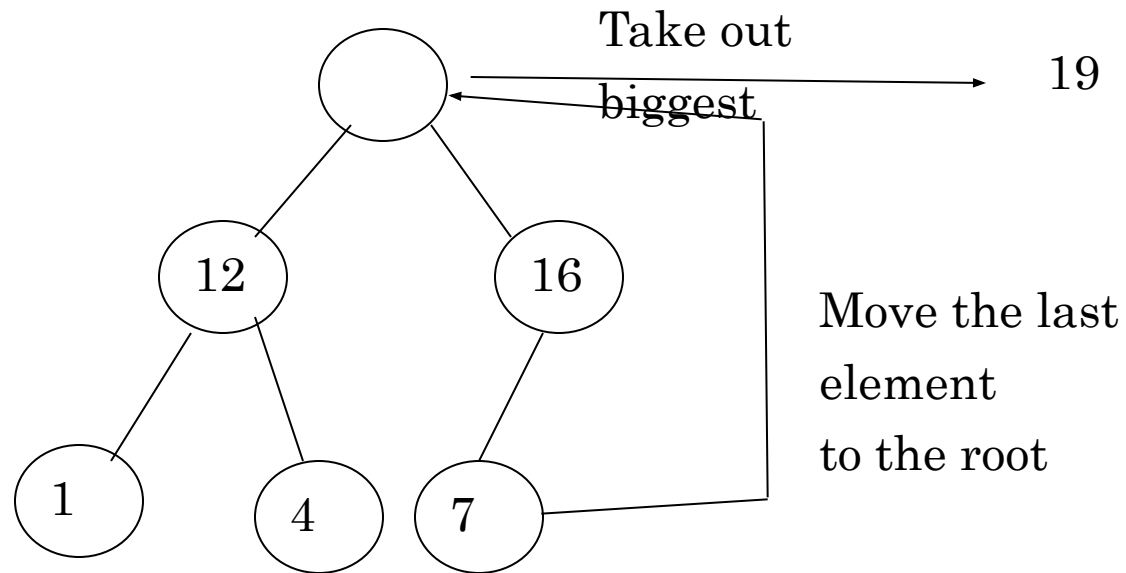
1. Set $\text{Item} := \text{Heap}[1]$ and Set $\text{Last} := \text{Heap}[N]$ and $N := N - 1$
2. Set $\text{Ptr} := 1$, $\text{Left} := 2$ and $\text{Right} := 3$
3. Repeat steps 5 to 6 while $\text{Right} \leq N$
4. If $\text{Last} \geq \text{Heap}[\text{Left}]$ and $\text{Last} \geq \text{Heap}[\text{Right}]$ then
Set $\text{Tree}[\text{Ptr}] := \text{Last}$ and Return.
5. If $\text{Heap}[\text{Right}] \leq \text{Heap}[\text{Left}]$ then
Set $\text{Heap}[\text{Ptr}] := \text{Heap}[\text{Left}]$ and $\text{Ptr} := \text{Left}$.
Else Set $\text{Heap}[\text{Ptr}] := \text{Heap}[\text{Right}]$ and $\text{Ptr} := \text{Right}$
6. Set $\text{Left} := 2 * \text{Ptr}$ and $\text{Right} := \text{Left} + 1$
1. If $\text{Left} = N$ and if $\text{Last} < \text{Heap}[\text{Left}]$ then Set $\text{Heap}[\text{Ptr}] := \text{Heap}[\text{Left}]$ and $\text{Ptr} := \text{Left}$.
2. Set $\text{Heap}[\text{Ptr}] := \text{Last}$
3. Return

Heap Sort

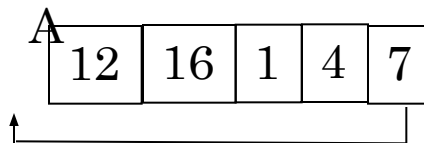
- The heapsort algorithm consists of two phases:
 - build a heap from an arbitrary array
 - use the heap to sort the data
- **To sort the elements in the decreasing order, use a min heap**
- **To sort the elements in the increasing order, use a max heap**



Example of Heap Sort

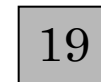


Array



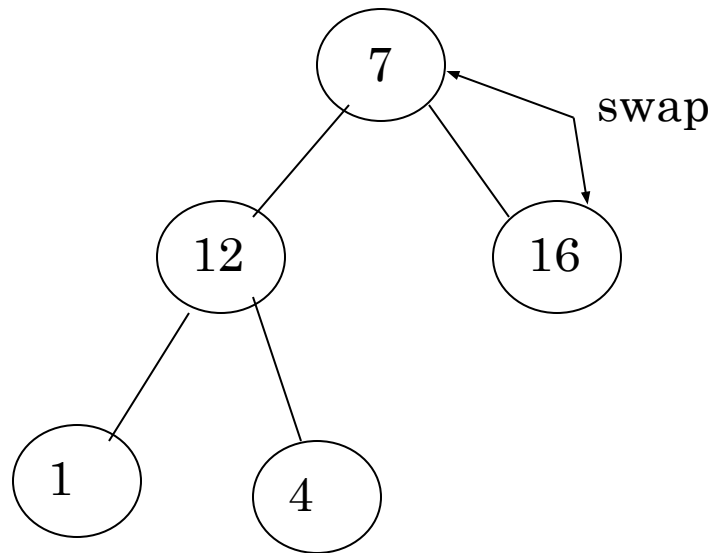
Sorted

:

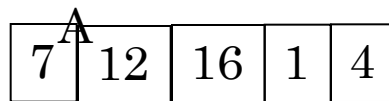


HEAPIFY

0



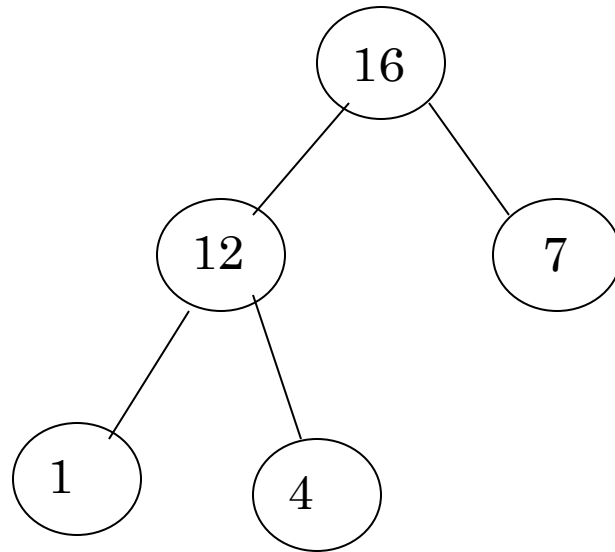
Array



Sorted

:

19



Array

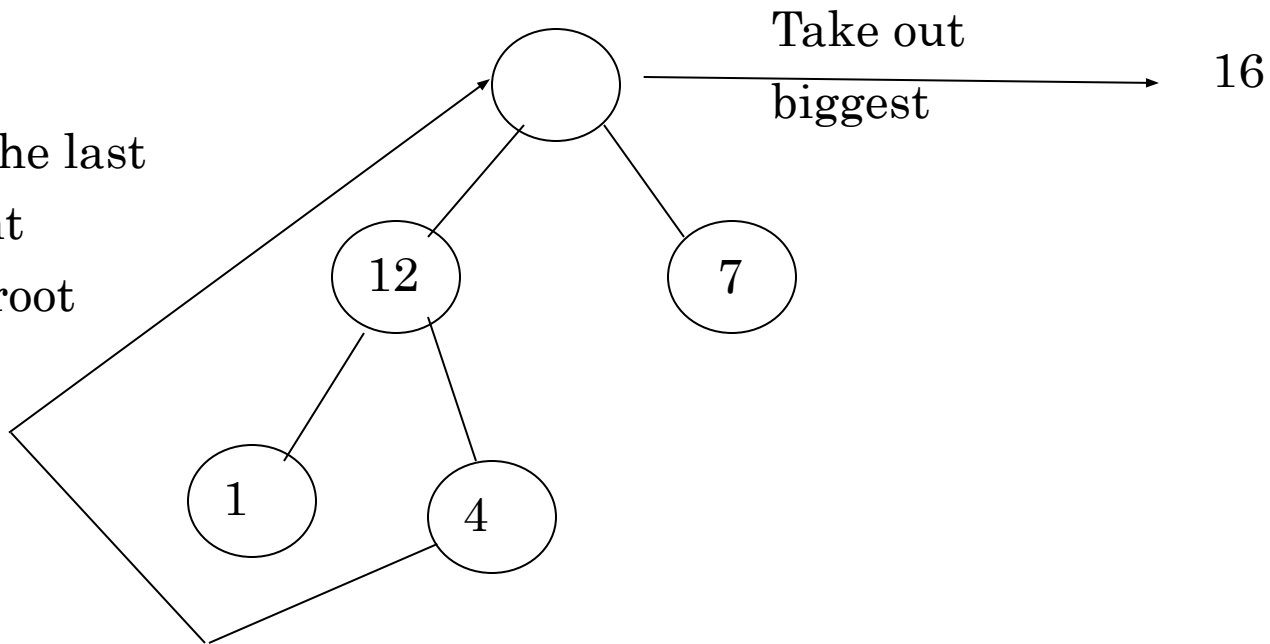
16	12	7	1	4
----	----	---	---	---

Sorted

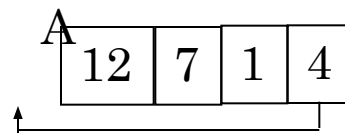
:

19

Move the last
element
to the root

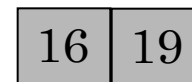


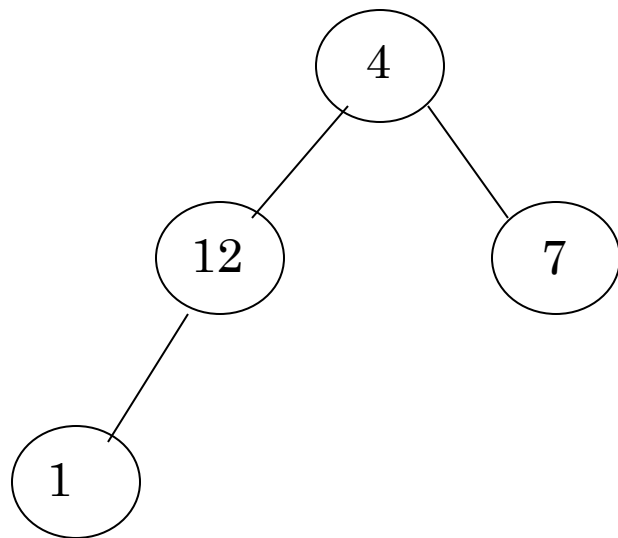
Array



Sorted

:





Array

4	12	7	1
---	----	---	---

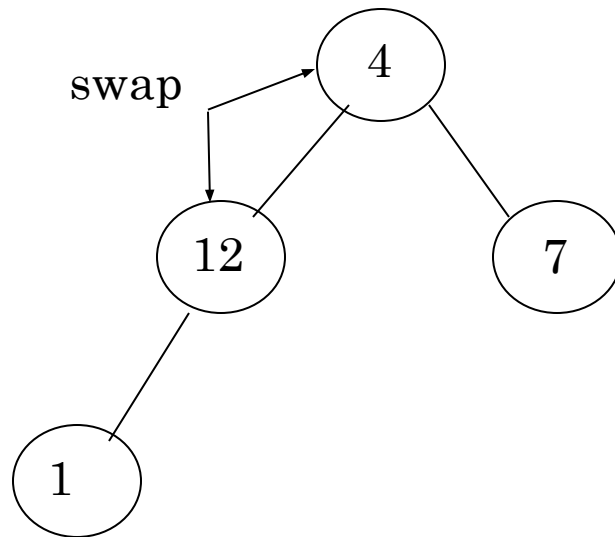
Sorted

:

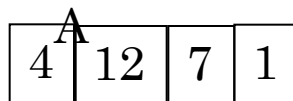
16	19
----	----

HEAPIFY

0

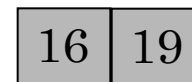


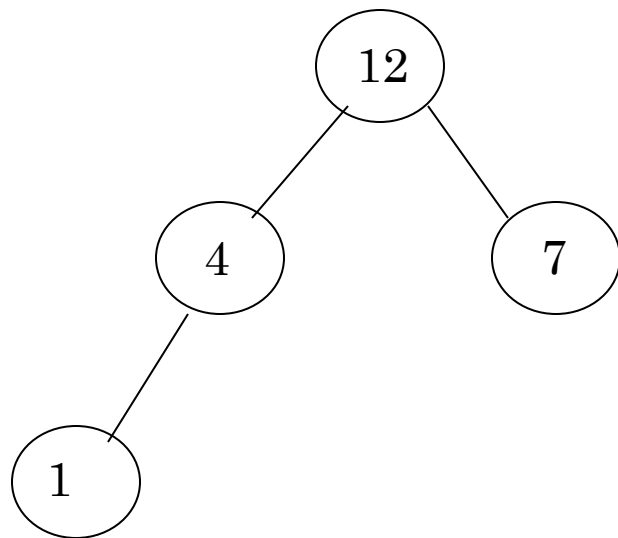
Array



Sorted

:





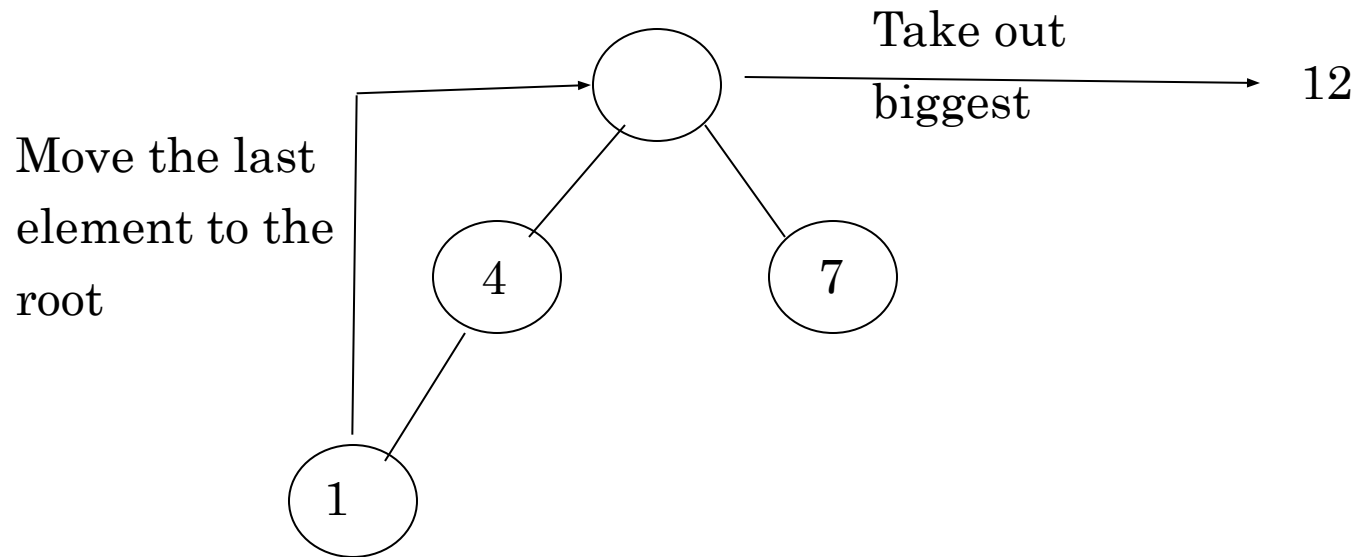
Array

A			
12	4	7	1

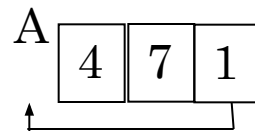
Sorted

:

16	19
----	----

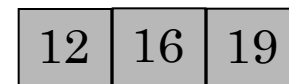


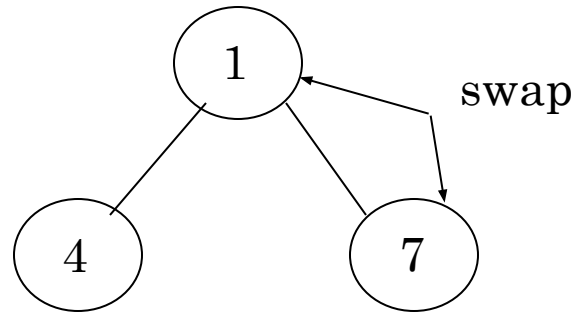
Array



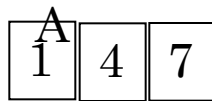
Sorted

:



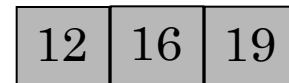


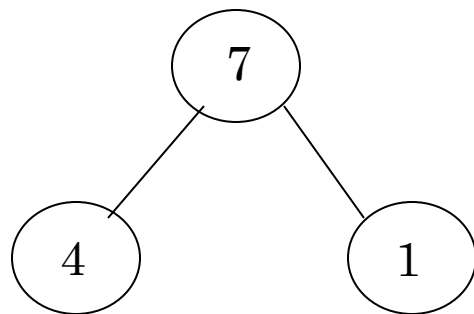
Array



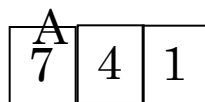
Sorted

:



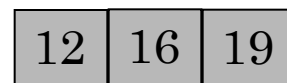


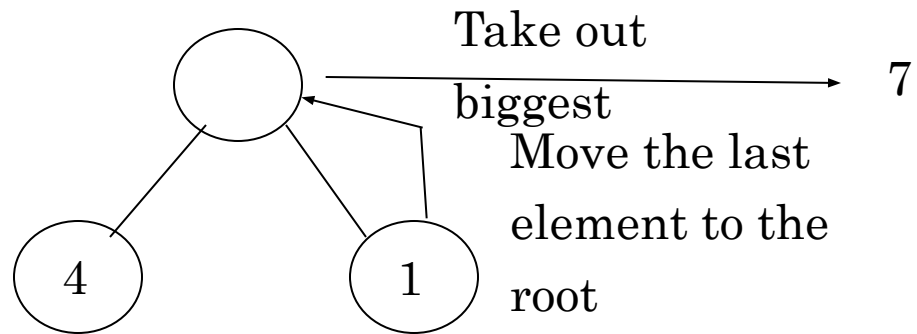
Array



Sorted

:





Array

A	
1	4

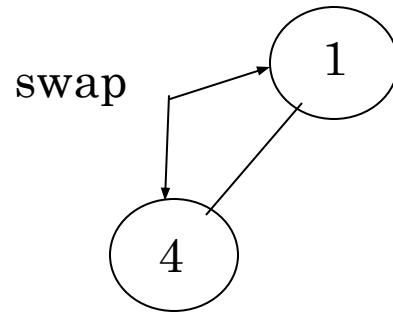
Sorted

:

7	12	16	19
---	----	----	----

HEAPIFY

()



Array

A

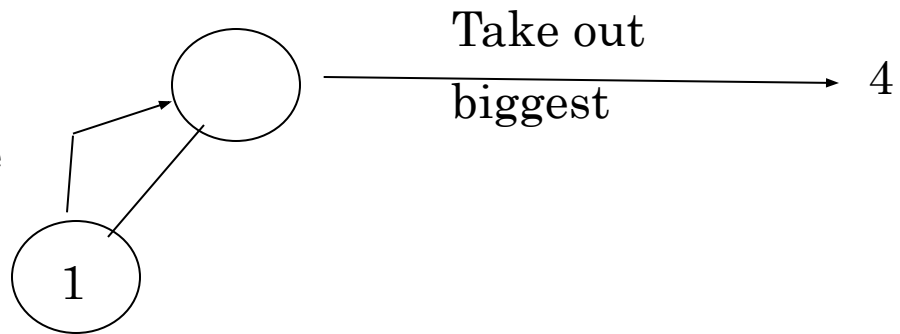
4	1
---	---

Sorted

:

7	12	16	19
---	----	----	----

Move the last
element to the
root



Array

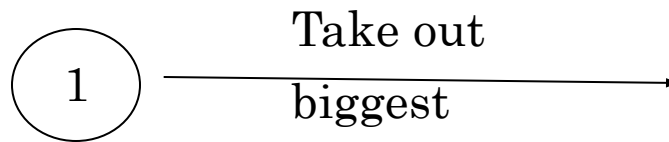
A

1

Sorted

:

4	7	12	16	19
---	---	----	----	----



Array

A

Sorted

:

1	4	7	12	16	19
---	---	---	----	----	----

Sorted

:

1	4	7	12	16	19
---	---	---	----	----	----

END!!!